

HPC2526 - Consegna 2: Performance dell'hardware

Le performance dei programmi paralleli dipendono dalle performance dei principali componenti hardware del sistema di calcolo: unità di processamento (CPU e GPU), accesso alla memoria, network, storage. In questo laboratorio verranno utilizzati semplici programmi di benchmarking dell'hardware, i cui risultati verranno presentati utilizzando i moduli Pandas e Matplotlib di Python.

Creare la directory HPC2526/2-Performance/ in cui copiate i file della consegna.

Sul cluster il tar file può essere copiato con il seguente comando:

```
cp /hpc/home/roberto.alfieri/SHARE/HPC2526/Performance.tgz .  
tar xzvf Performance.tgz
```

Performance.tgz contiene i file necessari per l'esecuzione sul PC e sul cluster:

- I sorgenti in linguaggio C (.c) sono i programmi di test.
- Gli script di shell (.sh) compilano i programmi, li eseguono sul PC e generano i risultati in un file csv (.csv).
- Gli script slurm (.slurm) compilano ed eseguono i programmi su cluster
- Gli script Python (.py) leggono gli output dei programmi di calcolo (file .csv) e generano i grafici in formato png.

Consegna: Generare i file csv, adattare e personalizzare gli script Python con Nome, Cognome e Data quindi generare il plot. Caricare i file csv e png che avete generato e scrivere una breve relazione sull'attività svolta in Relazione2.pdf o .txt

Nota: Nome, Cognome e Data dovranno essere personalizzati anche nei plot dei prossimi laboratori.

Attività da eseguire sul PC e/o sul cluster di Ateneo:

timing: Questo esercizio ha lo scopo di familiarizzare con le funzioni per determinare i tempi di calcolo, con il processo di esecuzione e con il postprocessing dei dati.

Nel programma timing.c la funzione clock_gettime() determina il tempo reale di esecuzione (CLOCK_REALTIME) e il tempo di utilizzo della sola CPU (CLOCK_PROCESS_CPUTIME_ID). Lo script timing.slurm può essere lanciato sia sul cluster (sbatch timing.slurm) che sul PC (sh timing.slurm) per compilare ed eseguire timing.c. Lo script timing.py legge il csv e genera il bar plot (timing.png).

MatrixMul: In un programma di moltiplicazione di due matrici quadrate ($n \times n$) il numero di operazioni floating point è: $\text{flops} = 2n^3$.

Possiamo quindi determinare la performance:

$\text{FLOP/s} = 2n^3 / T$ dove T è il tempo di esecuzione del programma.

Directory CPU:

- **mm.c** Calcolo matrixMul con algoritmo Naive oppure con funzione ottimizzata `gsl_blas_sgemm()` della libreria GSL (tramite opzione `-g`)
se necessario installare la libreria `gls`: **`apt update; apt-get install libgsl-dev`**
- **omp_mm.c** Versione Naive parallelizzata con openMP (`-t` seleziona il numero di thread)
- **mm.sh** Sul PC compila `mm.c` con `gcc` ed esegue sia Naive che GSL
- **mm.slurm** Sul cluster compila `mm.c` con `gcc` e con Intel, con [alcune opzioni di compilazione](#), ed esegue sia Naive che GSL.
- **omp_mm_scaling.slurm** Compila ed esegue `omp_mm.c` con un numero crescente di thread sia sul PC che sul cluster

Directory GPU:

- **matrixMul** Eseguitibile per la moltiplicazione di matrici su GPU: fa parte dei `cuda-samples` messi a disposizione da NVIDIA. Il programma calcola il numero di operazioni floating point, determina il tempo di esecuzione e stampa la performance in GFLOPs
- **matrixMul.slurm** Esegue `matrixMul` sia sul cluster (`sbatch matrixMul.slurm`) che sul PC (`sh matrixMul.slurm`).

Directory PLOTTING:

- **matrixMul.py** Dopo avere eseguito i programmi di test per il calcolo `matrixMul` in diversi contesti (PC, cluster, CPU, GPU, `gsl`, ecc) copiate i file `csv` nella directory PLOTTING. Lo script python `matrixMul.py` legge i file presenti e genera una plot che compara le diverse soluzioni.
Lo script python legge i dati nel seguente formato `csv`:
`N, gflop, tempo, GFLOPs, Hostname, COMMENT`
che è il formato di output `csv` di `mm.c` e `omp_mm.c`. Per la GPU il formato è diverso, non è un `csv`, e deve essere adattato manualmente editando il file.

Band-latency: Questi semplici benchmark (`bandwidth.c` e `latency.c`) determinano i valori di banda (MBps) e latenza (secondi) nella comunicazione tra due processi.

La Bandwidth (B) dipende della dimensione dei messaggi. La latenza è il tempo (L) necessario per il trasferimento del messaggio di dimensione minima.

Il tempo T per trasferire un messaggio di N byte è $T = L + N/B$

Il percorso fisico tra i processi può essere diverso a seconda della collocazione dei processi:

Memoria condivisa. Se i processi sono allocati sullo stesso nodo la comunicazione avviene tramite una area di memoria condivisa (Shared-Memory SHM) gestita dal sistema operativo.

Rete. Se i processi risiedono su nodi diversi la comunicazione avviene attraverso la rete che interconnette i nodi. Sul cluster la tecnologia di rete può essere Intel Omnipath (OPA) oppure Infiniband (IB).

I programmi bandwidth.c e latency.c vengono compilati ed eseguiti dagli script band-lat.sh (sul PC) e band-lat.slurm (sul cluster). Gli script python leggono i csv e generano i plot in formato png.

Disktest: Questo è un semplice benchmark per la valutazione delle prestazioni dello storage basato sul [comando dd](#).

sh disktest.sh determina le prestazioni di storage del PC.

sbatch disktest.slurm esegue il test in diverse partizioni dello storage del cluster : HOME, SCRATCH e ARCHIVE.